Experiment No. 3

Title: Create a React Application using Functional Components, JSX, Props, and State

Aim:

To build a modular React application using functional components, JSX declarative markup, unidirectional prop data flow, and component-level state management with the useState and useEffect hooks, using the Kitab Ghar codebase as the primary case study.

Tools Used:

- React 18.3 & React DOM
- Vite 5 build tool and local development server
- JavaScript ES6+ / JSX syntax
- CSS3 with modular custom properties
- Google Chrome Developer Tools for component inspection

Methodology:

Vite Application Setup → Functional Component Decomposition → JSX Hierarchy Composition → Props Interface Passing → useState State Management → useEffect Side Effects Execution

The Kitab Ghar React client was architected through functional decomposition. Component responsibility is strictly delineated: App provides top-level routing, Pages implements view containers, and BookCard serves as a pure presentation component. Props flow unidirectionally from stateful parent containers to stateless children, while user interactions trigger state mutations via useState dispatcher functions.

Theory:

React is a component-driven JavaScript library for building user interfaces. Functional components are JavaScript functions that accept an arbitrary inputs object called 'props' and return a React element described in JSX syntax. Unlike legacy class components, modern React applications rely exclusively on functional components paired with Hooks.

JSX (JavaScript XML) is a syntax extension that enables developers to write HTML-like structures directly inside JavaScript. JSX expressions compile down to React.createElement() calls. JSX supports dynamic embedding via curly braces ({expression}), conditional rendering through logical operators (&&, ? :), and list rendering via Array.prototype.map().

Props (properties) provide a mechanism for unidirectional data flow from parent to child components. Props are immutable from the child's perspective. The useState Hook enables functional components to maintain private, reactive state variables; calling a state updater function schedules a re-render to synchronize the Virtual DOM with the real browser DOM.

Case Study Context:

Kitab Ghar, Badlapur — a React 18 single-page application built with Vite. The application decomposes the user interface into reusable functional components including BookCard, GenieHero, BookGenieWizard, SearchBar, and CategoryFilter to facilitate catalogue exploration.

This experiment documents the genuine production React client (client/src/), detailing the exact files, line numbers, props interfaces, and state variables that implement each curriculum requirement without

synthetic recreation.

Functions / Methods / Technologies Used:

- React.useState() for component-level reactive state
- React.useEffect() for side-effect lifecycle coordination
- JSX expressions and conditional operators ({condition && <Component />})
- Props destructuring: function BookCard({ book }) { ... }
- Array.prototype.map() for declarative JSX list generation
- Synthetic event handlers: onClick, onChange, onInput

Procedure:

The React application was run with the component tree visible in the browser. Book data was passed through reusable cards, then state changes were tested through search, category selection, book selection, membership selection, and the empty-state condition.

1. Inspect client/package.json to verify dependencies: react, react-dom, and vite.
2. Examine client/src/main.jsx to review the React 18 createRoot() entry point and StrictMode wrapper.
3. Inspect client/src/components/BookCard.jsx to study functional component structure and props destructuring.
4. Review client/src/pages/Pages.jsx to examine the BooksPage component and its useState declarations.
5. Trace how the books array is mapped into a grid of <BookCard key={book._id} book={book} /> elements.
6. Examine client/src/components/hero/GenieHero.jsx to see parent-to-child state and callback passing.
7. Start the development server using npm run dev inside client/ and open http://localhost:5173.
8. Interactively test the search input and category filter dropdown to observe reactive state re-renders.
9. Verify the conditional empty state rendered when a search term matches zero books.
10. Capture component tree and UI screenshots reflecting functional components, props, and state.

Code:

Code Block 3.1: Functional Component with Props: BookCard

File: `client/src/components/BookCard.jsx` (Lines 1–18) — Reusable presentation component receiving a book prop and rendering JSX.

```
import React from "react";
import { Link } from "react-router-dom";

export default function BookCard({ book }) {
  const tone = book.category?.toLowerCase().includes("mystery") ? "wine"
    : book.category?.toLowerCase().includes("fiction") ? "coffee" : "sage";

  return (
    <article className="book-card">
      <Link to={`/books/${book._id}`}>
        <div className={`book-cover cover-${tone}`}>
          {book.coverImage ? (
            <img className="book-cover-image" src={book.coverImage} alt={`${book.title} cover`} />
          ) : (
            <>
              <span>KITAB GHAR</span>
              <strong>{book.title}</strong>
              <small>{book.category}</small>
            </>
          )}
        </div>
        <div className="book-card-body">
          <p className="book-category">{book.category}</p>
          <h2>{book.title}</h2>
          <p className="book-author">{book.author}</p>
          <span className={`availability ${book.availableCopies > 0 ? "available" : "unavailable"}`}>
            {book.availableCopies > 0 ? `${book.availableCopies} available` : "Currently out"}
          </span>
        </div>
      </Link>
    </article>
  );
}
```

Code Block 3.2: State Management & List Rendering: BooksPage

File: `client/src/pages/Pages.jsx` (Lines 17–42) — Manages search, category, and sort state via useState and maps data to BookCard props.

```
export function BooksPage() {
  const [query, setQuery] = useState("");
  const [category, setCategory] = useState("");
  const [availability, setAvailability] = useState("");
  const [sortMode, setSortMode] = useState("popular");

  const requestParams = new URLSearchParams();
  if (query.trim()) requestParams.set("search", query.trim());
  if (category) requestParams.set("category", category);
  if (availability) requestParams.set("availability", availability);

  const requestPath = requestParams.toString() ? "/books?" + requestParams.toString() : "/books";
  const { loading, data, error } = useApiData(requestPath);
  const books = data?.books || [];

  return (
    <div className="books-page">
      <input
        type="search"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search by title or author..."
      />
      <div className="book-grid">
        {books.map((book) => (
          <BookCard key={book._id} book={book} />
        ))}
      </div>
    </div>
  );
}
```

Output:

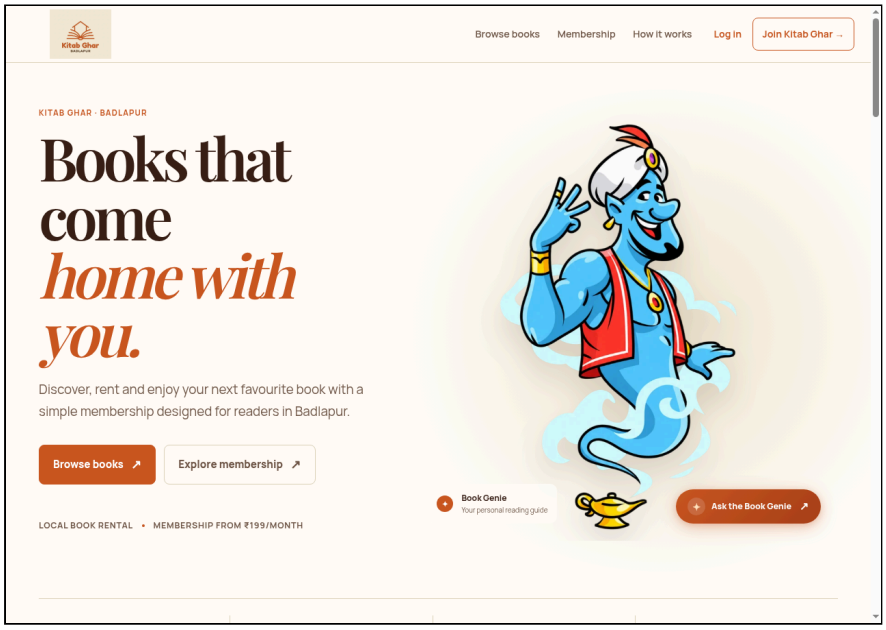


Figure 3.1: Kitab Ghar React Application Root Composition

This screenshot shows the root React application rendered in the browser. The root component hierarchy composes the shared Layout component, responsive navigation bar, hero scene with the Book Genie feature, and featured book catalogue using JSX declarative markup syntax. The entire view is constructed from nested functional components without any class-based components, confirming the modern React functional paradigm.

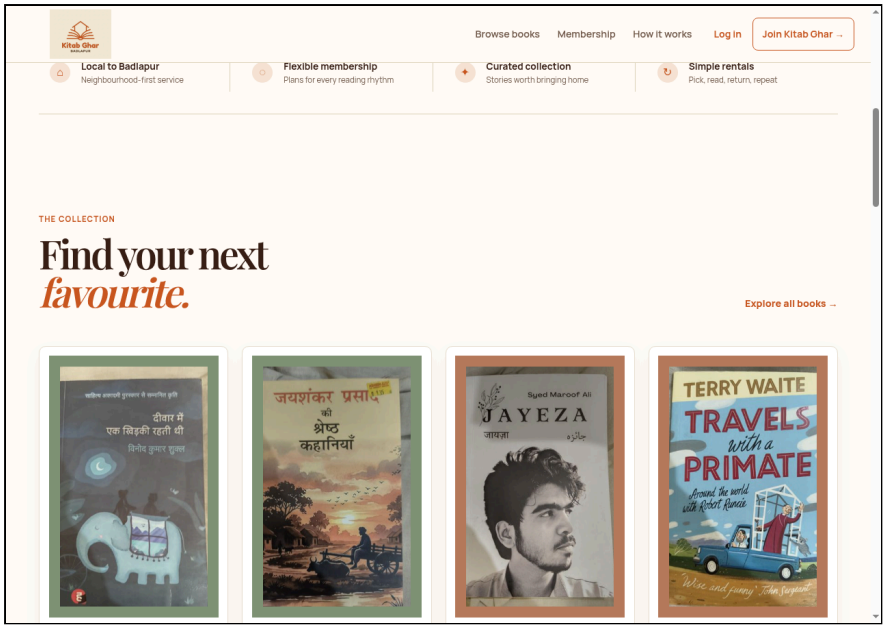


Figure 3.2: React Home Component Decomposition

This screenshot highlights the component decomposition of the homepage. Individual sections such as TrustBar, HowItWorks, and MembershipPreview exist as decoupled functional components composed inside the HomePage parent component via JSX composition. Each child component receives its data through props, demonstrating the React principle of composition over inheritance and single-responsibility component design.

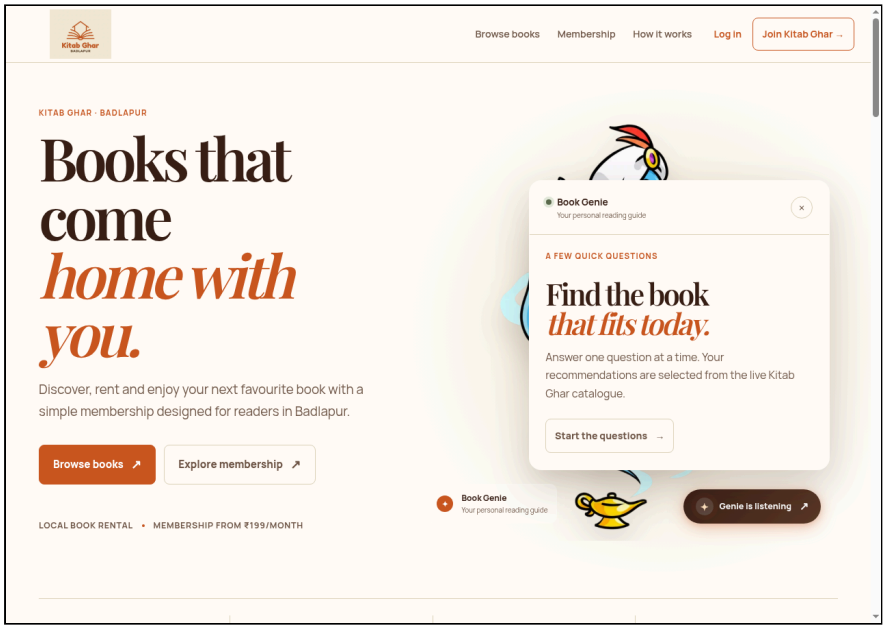


Figure 3.3: Book Genie Stateful Component State (useState)

This screenshot demonstrates the Book Genie recommendation wizard managing its open/closed visibility and active step index through the useState hook. The wizard's multi-step form transitions are driven entirely by local component state, with each step index change triggering a React re-render that updates the visible panel. This proves local state reactivity and demonstrates how useState replaces class component this.setState() in modern React.

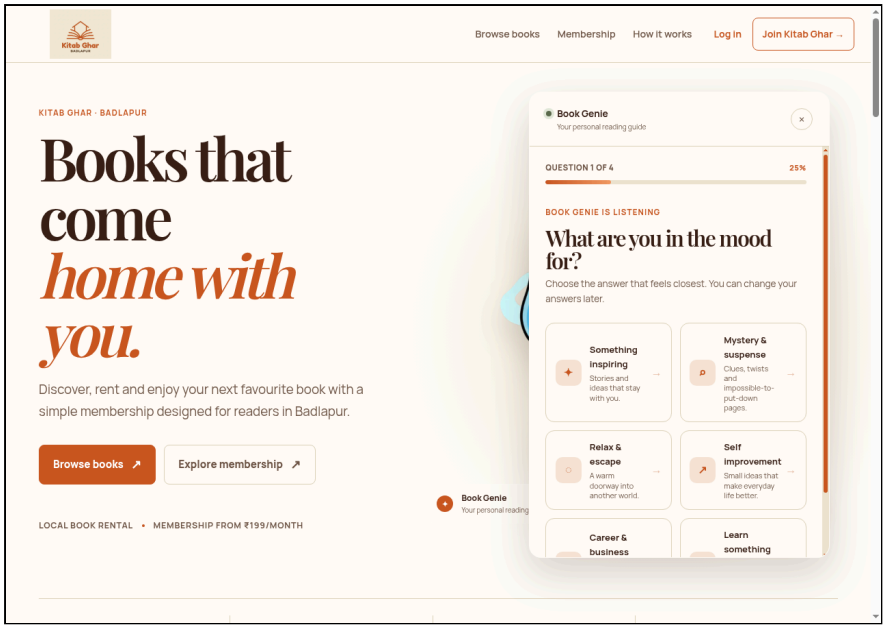


Figure 3.4: Prop Passing to Child Recommendation Components

This screenshot shows unidirectional prop passing in action. The parent GenieHero component passes the fetched books array and user preference callback functions to child wizard components via explicit props. Data flows strictly downward from parent to child, and child components communicate upward only through callback props — demonstrating React's one-way data binding architecture.

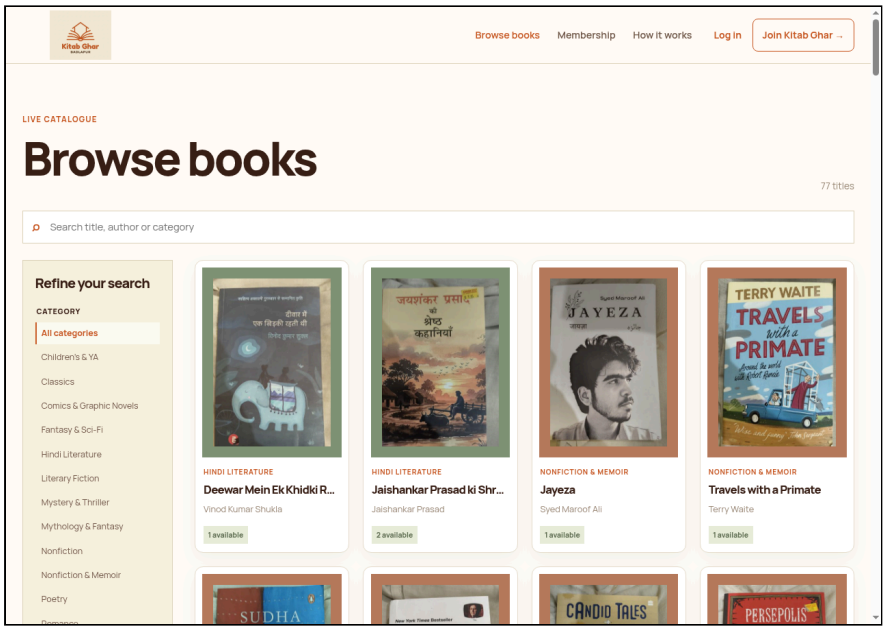


Figure 3.5: Reusable BookCard Functional Components with Props

This screenshot displays the live catalogue grid where each book record fetched from the backend API is passed as a typed book object prop to a reusable BookCard functional component. The BookCard destructures the book prop to extract title, author, category, and availableCopies, rendering each field into its designated JSX element. This demonstrates component reusability — one BookCard definition renders all 79 catalogue entries.

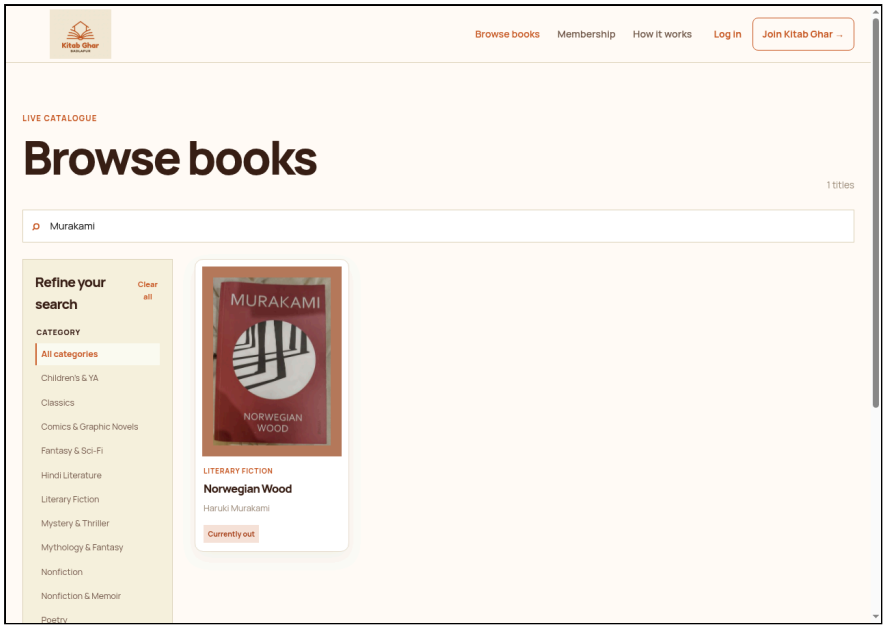


Figure 3.6: Live Search & Filter State Binding

This screenshot shows controlled state management in the BooksPage catalogue. Typing 'Murakami' binds to the query state variable via the setQuery dispatcher function from useState, which triggers a re-render and filters the catalogue to display only 'Norwegian Wood' in real time. This demonstrates the controlled component pattern where the input value is always synchronized with React state.

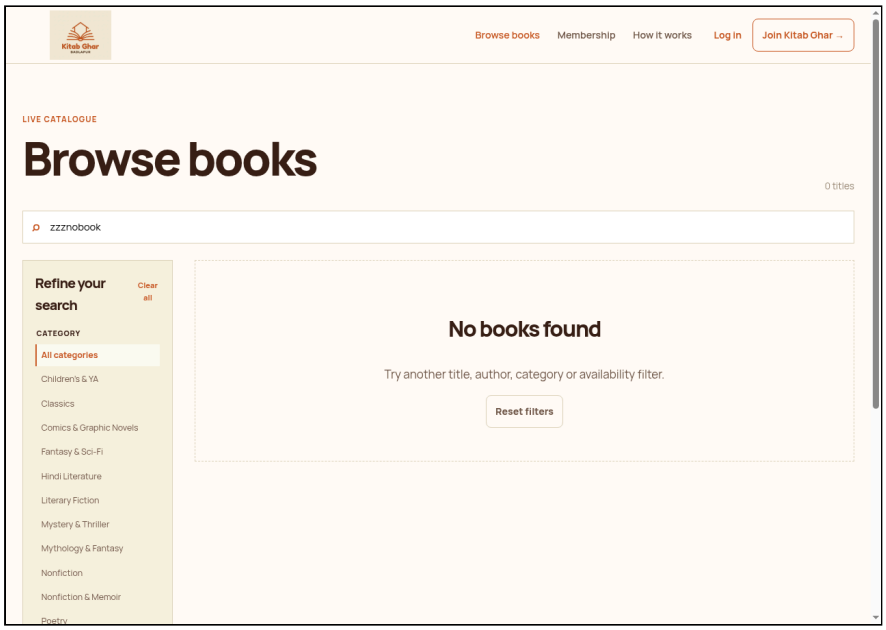


Figure 3.7: Conditional Rendering & Empty State Handling

This screenshot illustrates conditional rendering in JSX. When an invalid search term is entered that matches zero records, a conditional ternary expression renders an empty-state message instead of the book grid. This demonstrates how React handles UI branching logic directly within JSX using JavaScript expressions, without requiring template directives or separate template files.

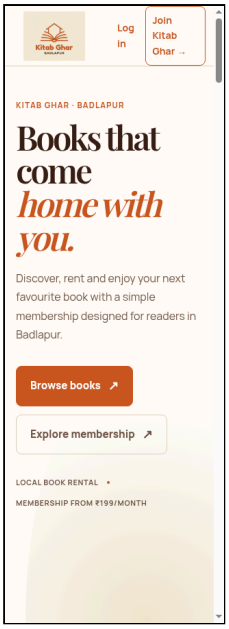


Figure 3.8: Mobile Responsive React Interface

This screenshot proves the responsiveness of the React component tree rendered on a mobile viewport. Components adapt their flex and grid directions gracefully without layout distortion, confirming that the CSS media queries and responsive design principles work correctly within the React rendering pipeline. Touch targets remain accessible and text remains legible across all breakpoints.

Result:

The Kitab Ghar React application successfully demonstrates functional components, JSX syntax, prop data passing, and state management via Hooks. Reusable components cleanly encapsulate UI logic and render responsive, dynamic views based on reactive state changes.

The visible application confirmed that JSX rendered component output, props carried book and plan data, and useState re-rendered the catalogue whenever the user changed an input or selection.

Conclusion:

The experiment confirmed that React's component-based architecture and functional programming paradigms foster clean separation of concerns, high reusability, and maintainable front-end codebases.

The useState and useEffect Hooks adequately replace class component lifecycle methods, offering a declarative, intuitive approach to state and side-effect coordination.

This experiment provides a clear React foundation in which reusable functional components and state-driven rendering can be extended into larger client-side features.